



**FACULTAD DE INGENIERÍA Y ARQUITECTURA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS**

BIG DATA

**DOCTOR DATA: PLATAFORMA DE PORTABILIDAD DE DATOS CLÍNICOS
CRÍTICOS CON ACCESO QR/NFC**

Autores

FERREYRA VILLARRIEL, Raúl Ricardo (ORCID: 0000-0002-9538-7951)

HUAPAYA ORMEÑO, Edgar Gerardo (ORCID: 0009-0001-2435-6453)

ORELLANO HUERTA, Hidgar (ORCID: 0000-0002-2791-9389)

PAREDES BECERRA, Anggie Marisol (ORCID: 0000-0002-4688-891X)

ZEVALLOS MACAVILCA, Ian Jesús (ORCID: 0000-0002-7629-0118)

Asesor

QUISPE LLACCTARIMAY, Ruben (ORCID: 0000-0003-2550-1840)

LIMA, PERÚ, JULIO DE 2026

1. Resumen

DoctorData es una plataforma de portabilidad de información clínica crítica orientada a mejorar la disponibilidad de datos relevantes durante una consulta o emergencia. El problema abordado es la fragmentación de información del paciente entre establecimientos, documentos personales y sistemas que no siempre pueden consultarse oportunamente.

El MVP integra un frontend web desarrollado con React y Vite, una API REST construida en Go con Gin, PostgreSQL como base de datos y un cliente móvil React Native. El backend concentra las reglas de negocio, la validación, el control de acceso por roles, el cifrado de campos sensibles y la auditoría. La infraestructura actual utiliza Cloudflare, servidores VPS Vultr y despliegue desde GitHub.

La propuesta de evolución plantea servicios gestionados en Azure, Kubernetes mediante AKS, PostgreSQL administrado, almacenamiento de archivos en Blob Storage, Firebase para capacidades móviles y automatización CI/CD. Esta arquitectura TO-BE constituye la meta de escalamiento; no se presenta como infraestructura ya desplegada.

Valor del proyecto. DoctorData busca reducir el tiempo necesario para acceder a información clínica esencial, facilitar la portabilidad controlada por el paciente y mantener trazabilidad sobre las operaciones sensibles. El sistema no reemplaza la historia clínica institucional: funciona como una capa portátil de datos críticos y acceso regulado.

2. Objetivo y alcance

2.1 Objetivo general

Desarrollar y evolucionar una solución tecnológica que permita registrar, proteger y consultar información clínica crítica del paciente, con acceso diferenciado según el rol y con una arquitectura capaz de escalar hacia servicios administrados.

2.2 Alcance presentado

AS-IS: situación actual	TO-BE: evolución propuesta
Frontend React + Vite y backend Go + Gin.	Azure AKS con pods separados para frontend y backend.
PostgreSQL alojado en una VPS privada.	Azure Database for PostgreSQL con Private Endpoint.
Cloudflare como DNS, proxy y acceso HTTPS.	CI/CD con Azure Repos y Azure Pipelines.
Despliegue manual desde GitHub.	Blob Storage y Firebase como servicios gestionados.

3. Arquitectura AS-IS

La arquitectura actual utiliza GoDaddy para el registro del dominio y delega el DNS a Cloudflare, que actúa como proxy HTTPS y gestiona el enrutamiento de correo. El frontend web React + Vite y el backend Go + Gin se ejecutan en una VPS Vultr. PostgreSQL opera en una segunda VPS comunicada mediante una red privada sin acceso público directo.

El código se conserva en GitHub y el despliegue vigente es manual mediante git pull o scripts SSH. La aplicación móvil React Native aparece como canal adicional, mientras que la evidencia principal del MVP corresponde a la plataforma web.

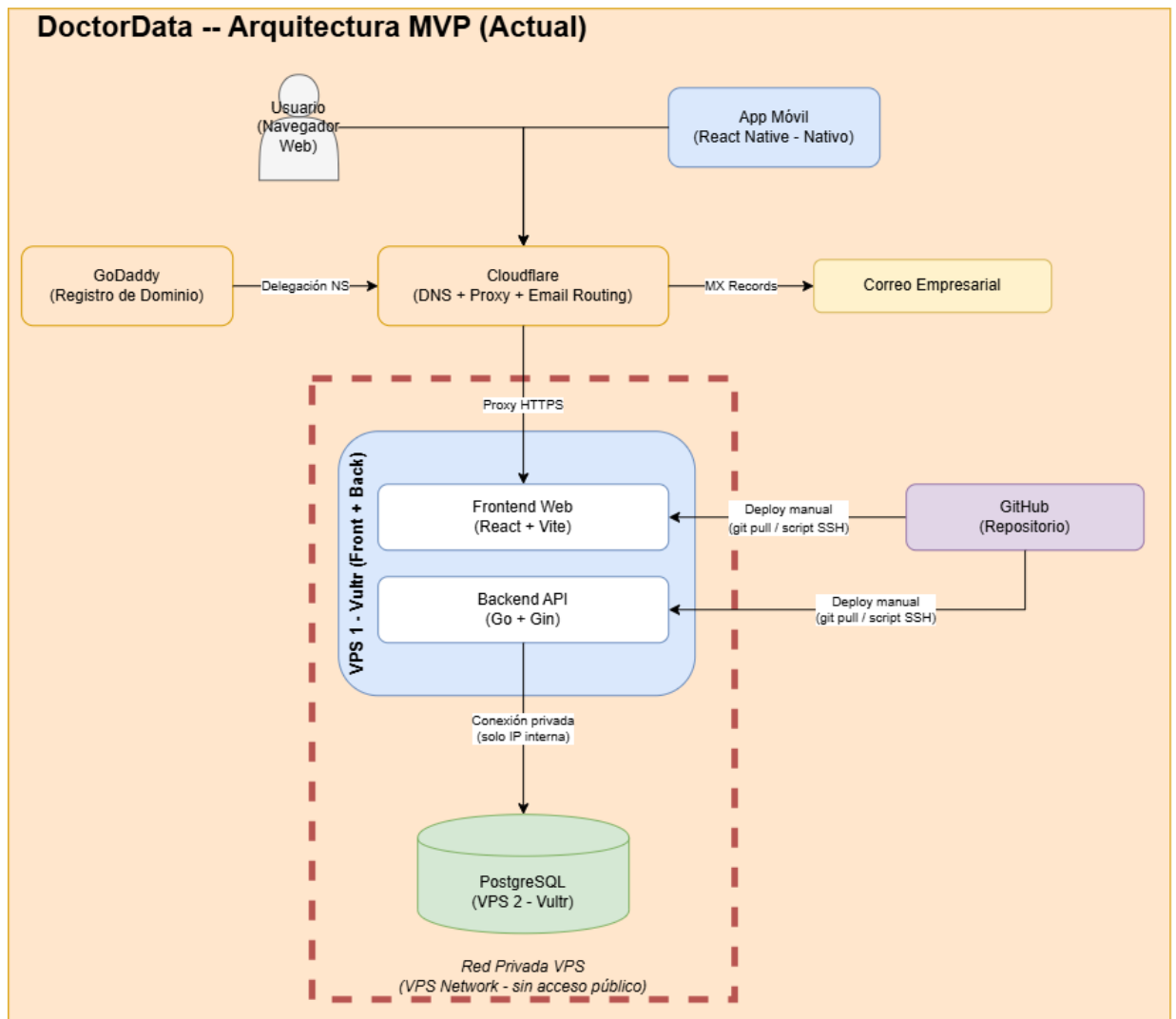


Figura 1. Arquitectura AS-IS del MVP DoctorData.

4. Casos de uso por rol

El modelo funcional separa las capacidades del paciente, doctor, administrador y superadministrador. El paciente administra sus datos y suscripción; el doctor consulta información autorizada y registra atenciones; el administrador valida profesionales y gestiona usuarios; el superadministrador hereda las funciones administrativas y controla roles y auditoría.

La separación por rol aplica el principio de mínimo privilegio. El acceso de emergencia mediante QR debe limitarse al conjunto mínimo de información permitido, sin convertir el perfil público en una vía de acceso al historial clínico completo.

DoctorData -- Casos de Uso por Rol

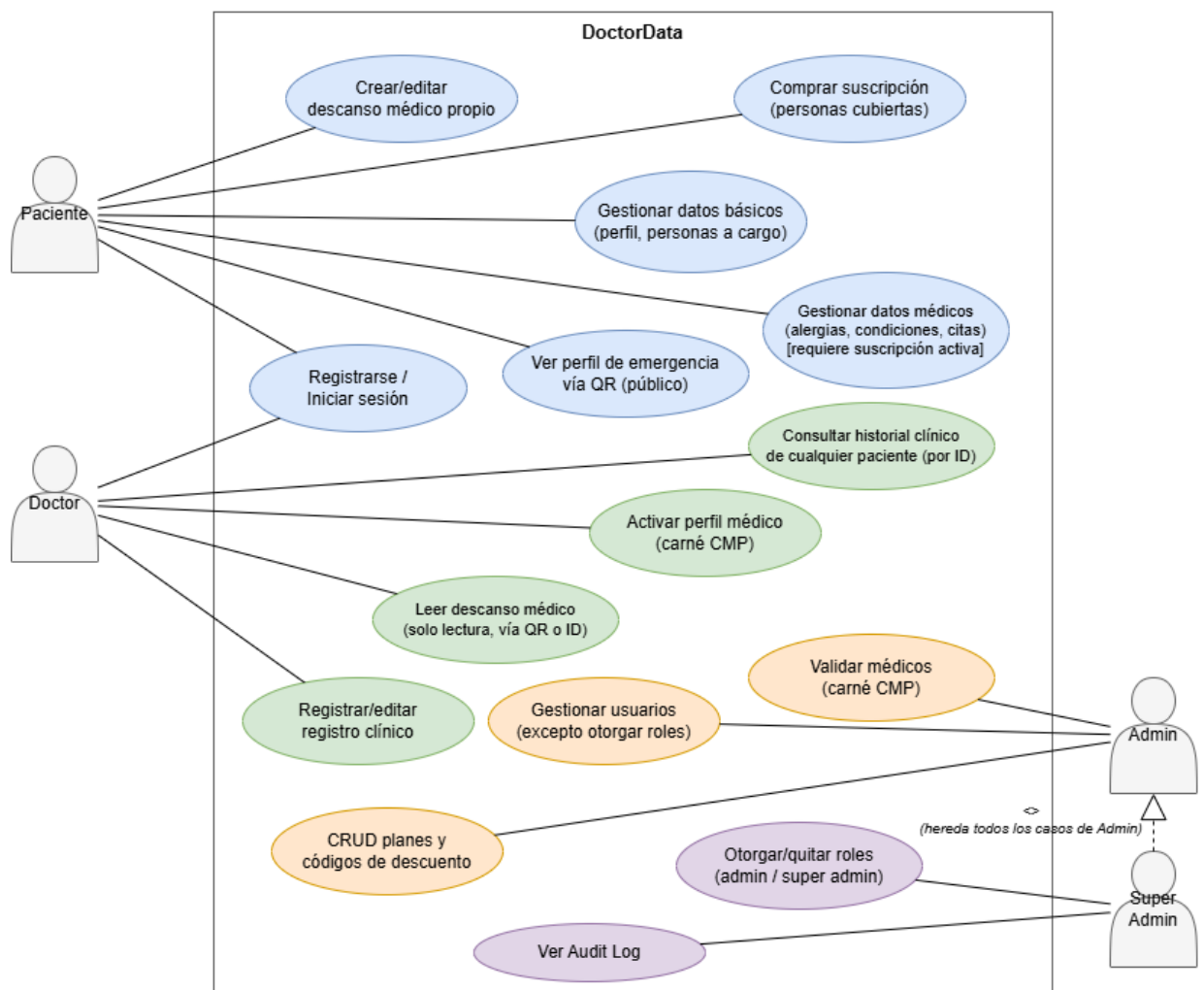


Figura 2. Casos de uso de DoctorData organizados por rol.

5. Flujo de datos: cifrado y sincronización

Las solicitudes web llegan al backend mediante HTTPS y JSON. Antes de crear o actualizar registros, el backend envuelve los campos sensibles en tipos cifrados. La clave de cifrado se obtiene de la variable de entorno `ENCRYPTION_KEY` y debe permanecer exclusivamente en el servidor. Los valores se protegen con AES-256-GCM y un nonce aleatorio por operación.

Los identificadores, llaves foráneas y campos necesarios para búsqueda u ordenamiento se mantienen en texto plano porque el cifrado aleatorio no permite comparaciones directas. La sincronización LAN del cliente móvil se encuentra señalada como endpoint pendiente. Asimismo, el diagrama registra el riesgo conocido de actualizaciones parciales cuando se envían valores incompletos; dicho comportamiento debe corregirse antes de producción.

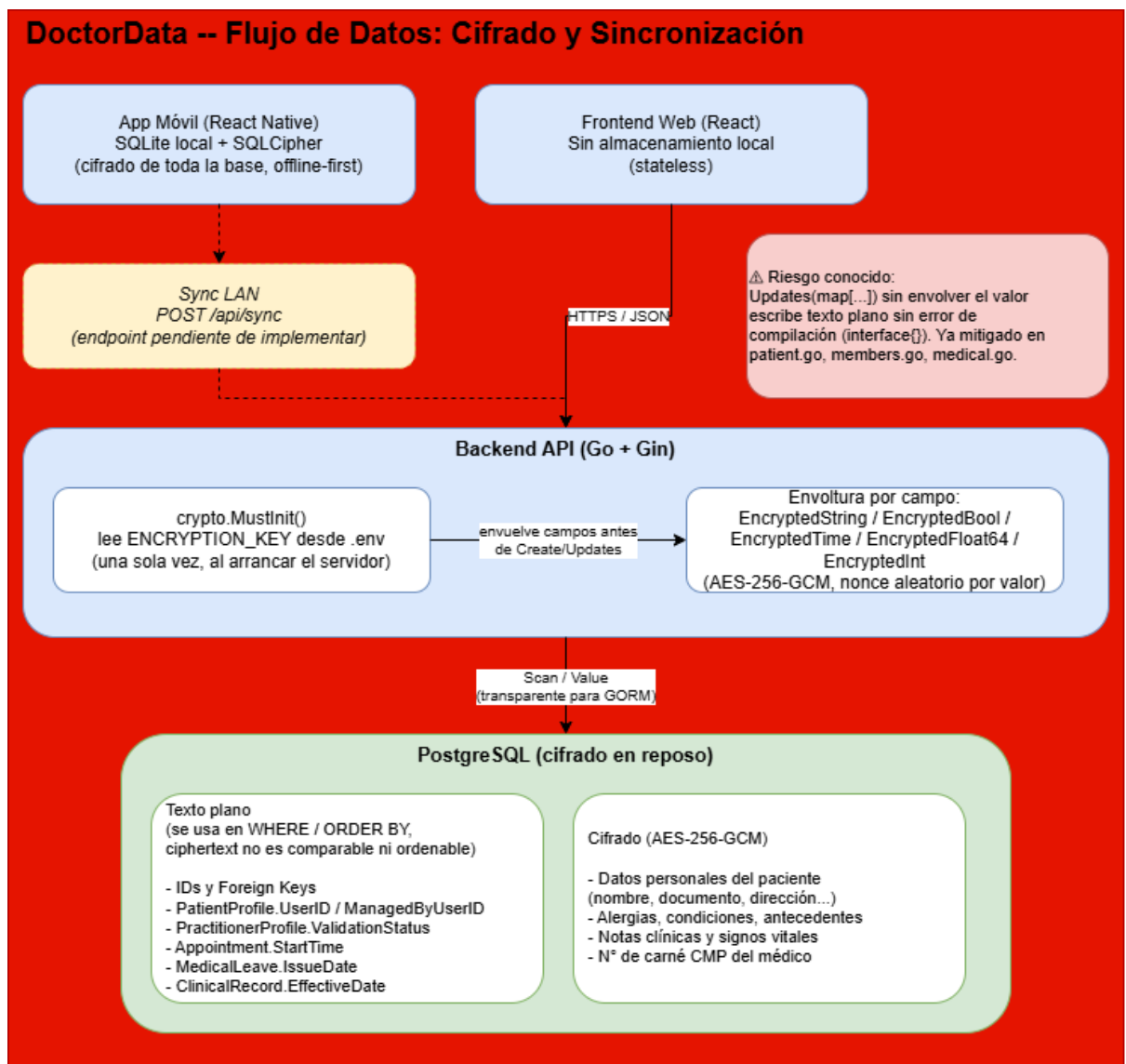


Figura 3. Flujo de cifrado, persistencia y sincronización.

6. Arquitectura TO-BE

La arquitectura objetivo conserva el dominio y Cloudflare, pero reemplaza el despliegue manual en VPS por un clúster Azure AKS. Un Ingress Controller distribuye las solicitudes hacia pods separados del frontend y del backend. Azure Database for PostgreSQL se conecta mediante una red virtual y Private Endpoint, evitando exposición pública de la base de datos.

Azure Blob Storage recibe archivos, mientras que Firebase aporta autenticación, mensajería FCM, Firestore y Maps para capacidades móviles. Azure Repos y Azure Pipelines automatizan integración y despliegue. Esta propuesta mejora escalabilidad, disponibilidad y trazabilidad operativa, pero también incrementa costos, complejidad y responsabilidades de gobierno.

DoctorData -- Arquitectura Objetivo (Versión Final)

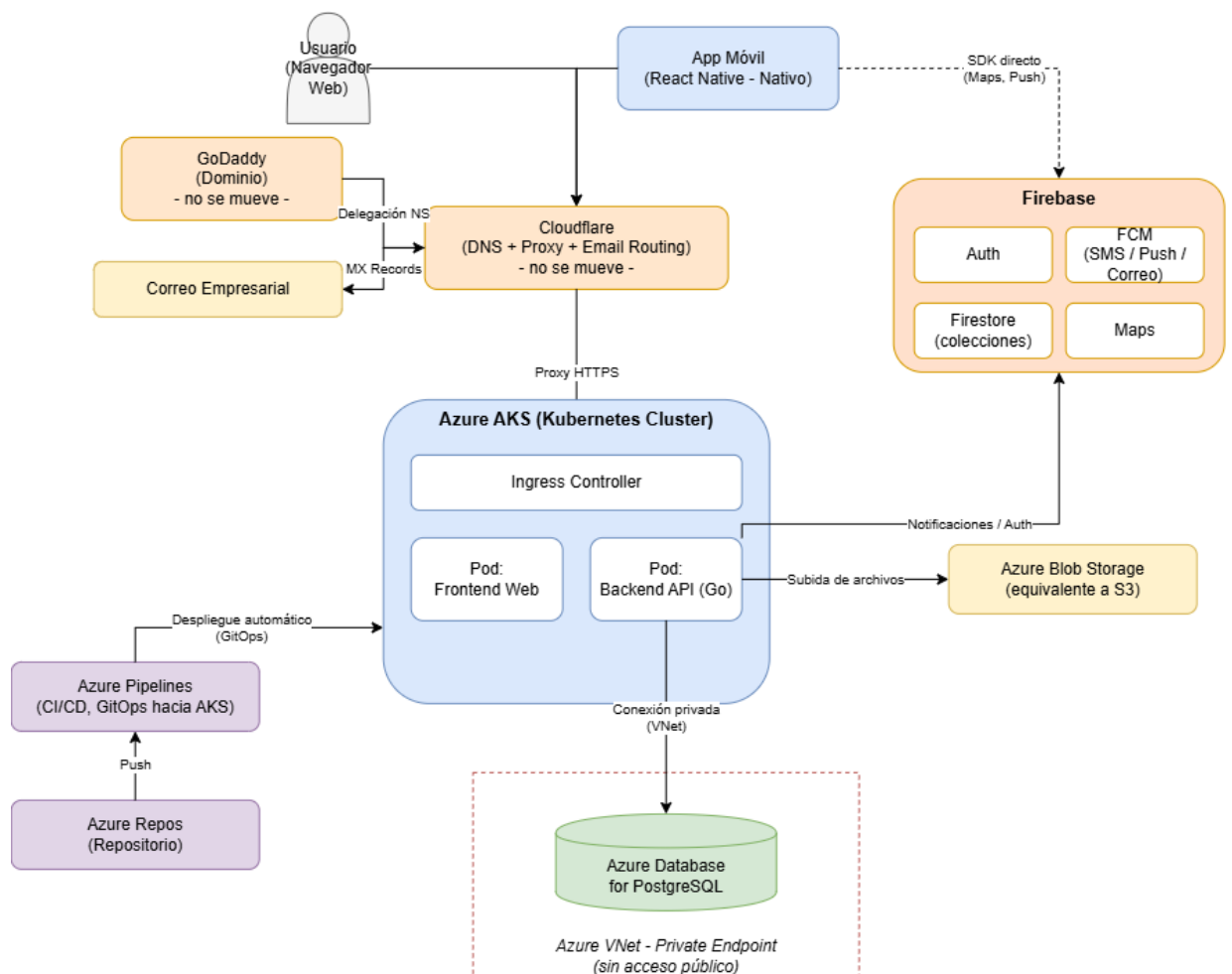


Figura 4. Arquitectura objetivo TO-BE de DoctorData.

7. Conclusiones

- DoctorData responde a un problema concreto de disponibilidad y fragmentación de información clínica crítica.
- La arquitectura AS-IS permite demostrar el MVP con React, Go, PostgreSQL, Cloudflare y VPS privadas.
- Los roles y la auditoría separan responsabilidades y reducen el acceso indiscriminado a datos sensibles.
- AES-256-GCM protege campos clínicos en la persistencia; la clave debe permanecer fuera del repositorio y del frontend.
- La sincronización móvil completa y la corrección de actualizaciones parciales permanecen como trabajo pendiente.
- La arquitectura TO-BE propone una evolución gradual hacia Azure AKS, servicios administrados y CI/CD, sin confundirla con el despliegue vigente.